Polypress

A lossless compressor for data tables. Measured against production binaries on real public data — not simulations, not estimates.

1.49×	73.7×	174 MB/s	exact
smaller than Parquet	compression, 278 MB file	decode throughput	round-trip fidelity

What it is

Polypress compresses tabular data — CSV, TSV, JSON, Parquet — and reproduces the logical table exactly: every column name, the column order, the row order, and every cell as the identical string. Every compression is verified in memory before anything is written to disk.

It beats every general-purpose compressor tested (xz, zstd, brotli, bzip2, gzip), the specialised numeric codecs shipped in ClickHouse (Gorilla, DoubleDelta, T64, Delta), the HPC array codecs zfp and fpzip, and Apache Parquet in four codecs — on every dataset tested.

The headline result

NHAMCS Adult, a CDC National Hospital Ambulatory Medical Care Survey extract: 96,539 rows × 209 columns, 277.9 MB. The full file, not a sample. Parquet reproduced the printed text of all 209 columns exactly here, so none of its size advantage comes from discarding formatting — this is a like-for-like comparison.

Codec	Bytes	Ratio	Time
Polypress	**3,956,941**	**73.65×**	**25 s**
parquet + brotli	5,881,047	49.55×	25 s
parquet + zstd	5,984,000	48.70×	21 s
zstd --ultra -22	6,178,417	47.17×	90 s
xz -9e	6,432,356	45.31×	28 s
brotli -q 11	7,132,152	40.86×	141 s
parquet + gzip	7,854,508	37.10×	1 s
bzip2 -9	8,989,156	32.42×	33 s
parquet + snappy	16,445,675	17.72×	0 s
gzip -9	18,781,409	15.52×	3 s

Parquet is the honest competitor: nobody stores a 209-column survey as compressed CSV. Polypress is **1.49× smaller than the best Parquet configuration**, at the same encode time.

Why it wins

Three ideas. The third is the one nothing else does.

1. Local function building

Fit a low-degree polynomial to the last few values in a column, extrapolate one step, store only the error. Because the fit is re-centred at every cell, the coefficients never have to be stored — the decoder already holds the neighbours. Order-*k* extrapolation turns out to be exactly the *k*-th finite difference.

2. The same idea in two dimensions

Where adjacent numeric columns are *commensurable* — same decimal places, same magnitude — a cell is predicted from its left, upper and upper-left neighbours. Detection matters as much as the predictor: differencing “Model Year” against “Make” is meaningless, so groups form only where columns are genuinely comparable.

3. Cross-column structure by reordering

Table columns are not independent — City is nearly determined by Postal Code. Rather than model that, Polypress **sorts the rows by the parent column**: equal parents become adjacent, the child collapses into long runs, and the entropy coder eats it. **The permutation costs nothing to store**, because the decoder has already rebuilt the parent and recomputes the same stable sort. Parents are chosen by conditional entropy with a Miller-Madow correction and arranged into a tree.

On the survey file, **172 of 200 dictionary columns were sorted by a parent**. Survey answers predict each other heavily, and no columnar format exploits this — Parquet compresses each column chunk independently by design. That is the entire margin.

Measured directly: on 40,000 rows of vehicle-registration data, coding City given Postal Code took 2,328 bytes by reordering versus 6,901 bytes using an adaptive context-modelling range coder — and reordering is far faster, because the work moves into C instead of a Python loop.

Results across every dataset tested

All from public sources. All verified exact round-trip.

Dataset	Shape	Polypress	Best other	Win
Treasury yield curve	7,003 × 8	21,981	44,148 xz	2.01×
Treasury + dates	7,003 × 9	26,574	50,572 xz	1.90×
WA EV population	289,564 × 16	2,262,193	3,917,084 xz	1.73×
NHAMCS survey (CDC)	96,539 × 209	3,956,941	5,881,047 pq	1.49×
EPA supply-chain GHG	18,288 × 8	79,166	116,900 xz	1.48×
LA crime (200k slice)	200,000 × 28	3,539,250	4,739,732 xz	1.34×

Against specialised numeric codecs

General-purpose tools were never the real competition for numeric tables. These are the shipped implementations — ClickHouse 26.8.1, and zfp/fpzip from the HPC world — on the Treasury yield curve (7,003 × 8).

Codec	Bytes	Ratio
Polypress	**21,981**	**13.07×**
ClickHouse Delta + ZSTD(22)	43,375	6.62×
zfp int32 lossless + xz	45,776	6.27×
ClickHouse DoubleDelta + ZSTD(22)	49,647	5.78×
ClickHouse T64 + ZSTD(22)	55,288	5.19×
ClickHouse Gorilla + ZSTD(22)	141,642	2.03×
fpzip lossless (float64)	255,606	1.12×

Every codec above predicts *down* a column. None predict *across* columns. Note the fair-comparison caveat: Gorilla, zfp and fpzip assume genuinely continuous floating-point physics data, so their float64 entries look poor on decimal-quantised financial values — the int32 rows are the fair ones, and Polypress still leads by 1.96×.

Speed

Encode beats every max-level general compressor. Decode is roughly comparable. Nothing here approaches the fast tier — zstd -3 encodes at 173 MB/s and always will — and this is still Python plus numpy with a small C library for the hot loops.

Dataset	Encode MB/s	Decode MB/s
NHAMCS survey	11.9	174
WA EV population	39.7	126
EPA supply-chain GHG	26.5	172
Treasury yield curve	21.7	85
— for reference: xz -9e	5.0	260
— for reference: brotli -q 11	1.1	516

A 278 MB file compresses in 25 seconds and restores in 1.4 seconds.

Files larger than memory

The single-shot codec holds the whole table as Python strings — a measured 8.1× expansion — so a 50 GB input would need hundreds of gigabytes and would not run. Polypress therefore also ships a block-streaming mode: the table is split into row blocks, each compressed independently behind an index, with peak memory set by the caller. The trade is explicit, because cross-column reordering only sees correlations inside a block.

Memory budget	Blocks	Ratio	Peak RSS
single-shot	1	73.65×	2.36 GB
2.0 GB	3	72.26×	1.89 GB
1.0 GB	6	69.12×	1.06 GB
0.25 GB	10	64.22×	0.64 GB

Blocks are independent, so this is also what makes parallelism possible. Projected for a 50 GB file at measured single-core throughput: about 95 minutes to compress, about 4 minutes to restore, with memory bounded at the chosen budget. That projection is arithmetic, not a measurement.

Polypress On-Demand — column random access

Parquet exists to read one column out of 209 and touch nothing else. The archival codec cannot, because its size win comes from *coupling* columns. That looked fatal for random access, and is not: the parent tree is shallow. On the 209-column survey the ancestor chain averages 2.78 and never exceeds 5, so reading one column costs about four column decodes rather than 209 — and each hop needs only an ancestor's ordering, never its string table.

Metric	Polypress On-Demand	Parquet
Archive size	4.2 MB (66.9×)	5.6 MB (49.6×)
Single-column read, 5 columns	0.061 s	0.055 s
Bytes touched per column	0.2–0.4 MB	—

1.33× smaller than Parquet, at within about 10% of its read speed, on the access pattern Parquet was designed for. This is a separate build from the archival codec and uses zstd rather than xz, because on-demand reads are dominated by decompression.

What is honestly not done

Stated plainly, because these decide whether this is a research result or a product.

- **One specimen at the top end.** The 73× result is a single survey file. It may be a property of that file. The next step is three or four more wide categorical tables — NHAMCS ED, NAMCS, BRFSS, NHANES — to establish whether the margin holds for a data class.
- **Python.** Encode is 9–40 MB/s. A C implementation is straightforward but unwritten; only the hot decimal/integer loops are C today.
- **Single-threaded.** Block independence makes parallelism easy, but it is not implemented.
- **No format specification, versioning policy, or fuzzing.** Nobody should license a format they cannot independently implement or trust with data they cannot re-create.
- **The predictors are prior art.** The planar predictor is Lorenzo (Ibarria et al., 2003, used in fpzip and SZ); MED is JPEG-LS; the range coder is LZMA's. What is not standard is the table-specific front end: commensurable-group detection and the reordering trick.

How the results were produced

Contenders are real binaries and real libraries, invoked at their strongest settings: `xz -9e`, `zstd --ultra -22`, `brotli -q 11`, `bzip2 -9`, `gzip -9`, pyarrow Parquet with zstd-22 / brotli-11 / gzip-9 / snappy, the ClickHouse 26.8.1 binary for Gorilla, DoubleDelta, T64 and Delta, and zfpy / fpzip from PyPI.

Two checks guard every number. First, fidelity is verified on both sides: Polypress round-trips are compared cell by cell, and Parquet was checked to confirm it was not winning size by silently discarding formatting. Second, an earlier version of this project claimed a 1% win over xz that turned out to be CSV quote-stripping rather than compression — feeding xz the same canonicalised bytes matched it to within 68 bytes. That claim was retracted, and the same control is now run before any comparison is reported.

Correctness is covered by 29 fidelity cases run twice, once through the C accelerator and once through the numpy fallback, on the principle that an accelerator which disagrees with the reference is not an accelerator but a second codec. Those tests caught three real defects that the benchmarks never would have: a 32-bit varint tail that truncated values past 2^{31}, newline-joined text storage that split any cell containing a newline, and a division by zero on an empty table.

Where this could be worth something

Cold archival of wide categorical tables — survey archives, regulatory retention, historical warehouse snapshots. Written once, read rarely, read whole. Storage is cheap, so a 1.5× improvement on a small warehouse is not a business; the case is strongest where data volumes are large, retention is mandatory, and the tables are wide and categorical. The On-Demand fork widens that to workloads that need column access, which is most analytics.

Datasets: catalog.data.gov (Treasury, WA EV population, EPA supply-chain GHG, LA crime), and a CDC NHAMCS extract. Benchmarks run on Apple silicon, macOS, Python 3.9. Every figure in this document is a measurement taken on that machine unless explicitly labelled a projection.